

: Custom Validation Rules

الـ **Validation** هو عملية بتأكد منها إن البيانات اللي المستخدم بيدخلها صحيحة قبل ما اخزنها في قاعدة البيانات .

طبعا **Laravel** موفرة ليا **Rules** جاهزة بس طبعا احيانا لا تكفي يكون محتاج اعمل **Rules** مخصصة وطبعا مش متوفرة في **Rules** جاهزة بناءا على احتياجاتي اللي انا هستخدمها فيها مثل : (تسجيل الدخول - تغيير كلمة مرور - رفع صورة - الخ)

ويتم انشاء **Rule** من خلال الامر ده مثال :

php artisan make:rule UsernameRule

```
1  class UsernameRule implements ValidationRule
2  {
3      public function validate(
4          string $attribute,
5          mixed $value,
6          Closure $fail
7      ): void {
8
9          if ($value == 'admin') {
10             $fail('هذا الاسم غير مسموح');
11         }
12     }
13 }
14
15 namespace App\Http\Controllers;
16
17 use Illuminate\Http\Request;
18
19 use App\Rules\UsernameRule;
20
21 class UserController extends Controller
22 {
23     //
24
25     public function store(Request $request)
26     {
27
28         $request->validate([
29             'username' => [
30                 'required',
31                 new UsernameRule(),
32             ]
33         ]);
34     }
35 }
```

دلوقتى عملت كلاس **Rule** واستخدمته فى ال **controller** لو كتبت فى اليوزر كلمة ادمن هيرجع ايرور وده :

وايضا هو ضحك حاجة فى حالة لو كانت القاعدة ستستخدم مرة واحدة فقط يستخدم **Closure** بدل من انشاء ملف

مثال :

```
1 'title' => [  
2  
3     function($attribute,$value,$fail){  
4  
5         if($value=="foo"){  
6  
7             $fail("Invalid");  
8  
9         }  
10  
11     }  
12  
13 ]
```

• Implicit Rules :

لارفيل فى حالة كتبت الحقل فاضي ولم تستخدم **required** يتجاهل معظم قواعد التحقق إذا كانت القيمة فارغة وفى حالة لو علوز ال **rule** تشتغل حتى لو القيمة فارغة وتستخدم الامر ده

php artisan make:rule Uppercase --implicit

وفى الحالة ده لارفيل بتستدعى **validate()** حتى عندما تكون القيمة فارغة وانت بداخل الميثود تحدد انك تقبل قيمة فارغة ولا ترجع **fail**

: Error Handling

لإدارة الأخطاء (Exceptions) وعرضها أو تسجيلها وظيفته التعامل مع الأخطاء بدون توقف الموقع بتاعى وليكن بدل مثلا يظهر للمستخدم حصل **fatal error** ممكن اظهره حصل خطأ حاول مرة أخرى وطبعا بسجل الخطأ فى ملف **log** . ومن اهم الاعدادات ده في ملف **bootstrap/app.php** باستخدام ميثود **withExceptions()**

وايضا في اعداد الكونفجريشن بحدد المتغير **APP_DEBUG** بحيث لو في مرحلة التطوير لسه بخليه **true** ولو خلاص طلع **production** بخليه **false** .

١. الإبلاغ عن الأخطاء (Reporting Exceptions) :

لارفيل ببسجل الأخطاء تلقائيا ويبيع لملفات **log** او يرسلها لخدمات زى (**Bugsnag - Flare - Sentry**) من خلال الامر ده **\$exceptions->report(...)**

وايضا لو حابب ان ازود بيانات اضافية مثل : بيانات عامة او **user_id** تظهر مع كل **log** ببستخدم الميثود ده :

\$exceptions->context()

٢. report() Helper :

ده ميثود من خلال لو عاوز اسجل الخطا بدون وقف تشغيل البرنامج بتاعى .

٣. منع تكرار تسجيل نفس الخطأ :

في حالة تم تكرار نفس الخطا وعاوزه يتسجل مرة واحدة حتى لو استخدم **report** اكثر من مرة ببستخدم الامر ده :

\$exceptions->dontReportDuplicates()

لارفيل بيتجاهل بعض الأخطاء مثل : (**403 - 404 - 419**) ولو عاوزه يرجع يسجلها ببستخدم الميثود ده **stopIgnoring()**

٤. Log Levels :

تحديد مستوى اهمية الخطا من خلال الامر ده

\$exceptions->level(PDOException::class, LogLevel::CRITICAL)

❖ أشهر المستويات:

- DEBUG
- INFO
- WARNING
- ERROR
- CRITICAL

٥. في حالة لو عاوز اتجاهل نوع معين من الخطا :
بستخدم الامر ده : `dontReport()` او لو في شروط معينة باستخدام الامر ده
`dontReportWhen(...)`

٦. عرض الأخطاء (Rendering) :
وده من خلاله بخصص المستخدم بتاعى يشوف ايه واعرضله في صورة أي
سواء كان (صفحة blade - ارسال json)

من خلال الامر ده : `$exceptions->render (.....)`
و Laravel يحدد تلقائياً نوع الرد حسب Accept Header اما في حالة
لو عاوز اخصه باستخدام ميثود ده `shouldRenderJsonWhen()`

٧. اخصص response بالكامل :
مثال عند الخطأ 419 يرجع المستخدم للصفحة السابقة مع رسالة مناسبة
باستخدام الميثود ده `respond(...)`

٨. تقليل عدد الأخطاء المرسله (Throttling) :
لو التطبيق عندي بيطلع ايرور كبيرة جدا ممكن اتحكم فيها انه اسجل اى واحدله
من خلال الامر ده

`Lottery::odds(1,1000)`

٩. HTTP Exceptions :
لارجاع خطا http بسهولة باستخدام الميثود ده `abort()` واكتب جواها
رقم الخطا
وفى الحالة ده ممكن اعمل ملفات جوه `resources/views/errors`
مخصصة لكل ايرور لارفيل بيعرضها تلقائيا